

AI PLACEMENT MANAGEMENT SYSTEM

Requirement Analysis Document

ASP.NET Core MVC + Web API + SQL Server + AI/ML

Team Members

Group – 12 Batch (8B)

IDs	Name
IN26012701	Urmi Barman
IN26014091	Sanskriti Singh
IN26014131	Shriya Punnal
IN26013408	Saaksham Kindra
IN26014085	Anshika Srivastava
IN26012858	Punit Kumar

1. Introduction

The AI Placement Management System is a web-based platform intended to digitize and streamline the college placement process. It provides a centralized environment connecting students, recruiters, and placement administrators. The system combines conventional placement-management workflows with AI-based resume analysis and job-matching capabilities.

The proposed solution uses ASP.NET Core MVC for the user interface, ASP.NET Core Web API for backend services, Entity Framework Core for data access, Microsoft SQL Server for persistent storage, and a separate Python/FastAPI AI service for intelligent features.

The system is designed as one integrated product rather than a collection of disconnected CRUD modules. A complete workflow should cover student registration, resume analysis, job creation, eligibility checking, AI matching, application, recruitment stages, interview results, selection, and administrative placement reporting.

2. Problem Statement

College placement activities often involve multiple disconnected records, manual resume screening, fragmented application tracking, and limited visibility into placement statistics. Students may find it difficult to identify suitable opportunities, while recruiters may spend significant time reviewing candidate information. Placement administrators require a consolidated view of students, companies, drives, applications, and final placement outcomes.

The proposed system addresses these issues by centralizing placement data and introducing AI-assisted resume analysis, semantic job matching, and skill-gap identification.

2.1 Proposed Solution

- Provide role-based access for Student, Company/Recruiter, and Admin users.
- Maintain student profiles, academic information, skills, certifications, projects, and resumes.
- Allow companies to create and manage job drives with eligibility criteria and required skills.
- Allow administrators to approve or reject job drives and monitor system-wide activity.
- Use AI to extract relevant skills from resumes and compare student profiles with job requirements.
- Provide match scores and skill-gap information to support student and recruiter decisions.
- Track applications from application through assessment/interview to selection or rejection.
- Maintain placement results and provide analytics and reports.

3. Objectives

- Centralize the college placement process in a single web application.
- Provide secure role-based access for students, recruiters, and administrators.
- Reduce manual effort in resume screening and candidate-job comparison.
- Provide AI-based resume skill extraction, compatibility scoring, and skill-gap analysis.
- Improve transparency by tracking job drives, applications, interviews, and selection outcomes.
- Provide administrators with dashboards and analytical summaries based on stored placement data.
- Create a maintainable full-stack architecture that supports API-based integration and automated testing.

4. Project Scope

The project is divided into three coordinated modules. Each pair is responsible for both frontend and backend work within the shared application.

Module	Main Scope
Student + AI	Registration/profile, academic details, skills, certifications, projects, resume upload/versioning, jobs, applications, AI recommendations, match score, and skill gaps.
Company + Recruitment	Company profile, job-drive creation/editing/publishing/closing, eligibility criteria, applicant review, shortlisting/rejection, assessment and interview scheduling, recruitment pipeline.
Admin + Analytics	Admin authentication, student/company management, job-drive approval/rejection, application and placement monitoring, dashboard, analytics, reports, integration and testing coordination.

4.1 In Scope

- Web-based placement management application.
- Role-based authentication and authorization.
- Student, company, job, application, interview, and placement data management.
- AI-assisted resume and job analysis.
- Dashboard and analytics.
- REST APIs and database integration.
- Unit, API, integration, and UI-oriented testing.

4.2 Out of Scope / Not Defined Yet

- External job portals and third-party recruitment platform integrations.
- Automated offer-letter generation unless added during later requirements refinement.
- Production cloud deployment architecture and production infrastructure sizing.
- Any AI capability beyond resume analysis, matching, and skill-gap analysis unless separately approved.

5. Stakeholders and User Roles

Role	Responsibilities / Needs
Student	Create profile, upload resume, view jobs, receive AI recommendations, apply, track applications and interviews.
Company / Recruiter	Create job drives, define requirements, view applicants, use AI match scores, shortlist candidates, manage interviews.
Placement Admin	Manage students and companies, approve drives, monitor applications and placements, view analytics and reports.
Faculty / Project Guide	Review requirements, implementation progress, testing, documentation, and final demonstration.

6. Functional Requirements

The following requirements define the expected system behavior. Requirement IDs can be used later for API endpoints, test cases, and traceability.

ID	Requirement	Priority
FR-01	The system shall allow users to register and authenticate according to their assigned role.	High
FR-02	The system shall enforce role-based authorization for Student, Company/Recruiter, and Admin functions.	High
FR-03	The system shall allow students to create and maintain their academic and professional profile.	High
FR-04	The system shall allow students to upload and manage resumes.	High
FR-05	The system shall analyze resumes and extract relevant skills and useful profile information.	High
FR-06	The system shall allow students to view available job drives and job details.	High
FR-07	The system shall allow companies to create, edit, publish, and close job drives.	High
FR-08	The system shall store minimum CGPA, eligible branches, backlog rules, graduation year, and required skills for a job drive.	High
FR-09	The system shall check student eligibility against job-drive criteria.	High
FR-10	The system shall compare student resume/profile information with job descriptions using AI-based semantic matching.	High
FR-11	The system shall calculate and display a compatibility/match score.	High
FR-12	The system shall identify skill gaps for a selected job.	High
FR-13	The system shall allow eligible students to apply for jobs.	High
FR-14	The system shall track application status through defined recruitment stages.	High
FR-15	The system shall allow recruiters to view applicants and AI match scores.	High
FR-16	The system shall allow recruiters to shortlist or reject candidates and schedule interview rounds.	High
FR-17	The system shall allow admins to manage students and companies.	High
FR-18	The system shall allow admins to approve or reject job drives.	High
FR-19	The system shall allow admins to monitor applications and placement outcomes.	High
FR-20	The system shall provide an admin dashboard containing key placement statistics.	High
FR-21	The system shall provide analytics for placement percentage, company/job applications, branch-wise placements, average package, and highest package.	High
FR-22	The system shall store interview results and final placement results.	High
FR-23	The system shall expose backend functionality through REST APIs.	High
FR-24	The system shall support system-wide testing and integration across the three modules.	Medium

7. Major Business Workflows

7.1 Student-to-Placement Workflow

Student registers → completes profile → uploads resume → AI analyzes resume → company creates job drive → system checks eligibility → AI calculates match score and skill gaps → student applies → recruiter reviews → candidate is shortlisted → assessment/interview → result recorded → student is selected/rejected → admin dashboard reflects placement outcome.

7.2 Recruitment Pipeline

Applied → Under Review → Shortlisted → Assessment → Technical Interview → HR Interview → Selected / Rejected

7.3 Admin Workflow

Admin logs in → reviews system dashboard → manages students/companies → reviews pending job drives → approves/rejects drives → monitors applications → records/monitors placement outcomes → reviews analytics and reports.

8. Data Requirements

The shared database should support the following core entities identified for the project:

Data Area	Core Entities
Authentication	Users, Roles
Student	StudentProfiles, Skills, StudentSkills, Resumes, Certifications, Projects
Company / Jobs	CompanyProfiles, JobDrives, JobSkills, EligibilityCriteria
Applications	Applications, ApplicationStatusHistory
Interviews	InterviewRounds, InterviewSchedules, InterviewResults
Placements	PlacementResults
AI / ML	ResumeAnalysis, ExtractedSkills, JobMatchScores, SkillGaps, AIRecommendations

9. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	Passwords and authentication credentials shall be handled using secure authentication mechanisms; access shall be restricted by role.
NFR-02	Authorization	Admin, Student, and Company functions shall not be accessible to unauthorized roles.
NFR-03	Performance	Normal CRUD and dashboard requests should return within a reasonable interactive response time under expected project-demo load.
NFR-04	Reliability	Database operations shall handle validation and failures without silently losing placement data.
NFR-05	Maintainability	The system shall follow a layered architecture with separated MVC, API, core, infrastructure, AI, and testing concerns.

NFR-06	Scalability	The API and database design should allow additional students, companies, drives, and applications without redesigning the complete system.
NFR-07	Usability	The UI shall use a common design system and provide clear navigation for each role.
NFR-08	Testability	Core functionality shall be testable using unit, API, integration, and UI-oriented tests.
NFR-09	Compatibility	The web application shall work in modern desktop browsers used for project demonstration.
NFR-10	Consistency	All three modules shall use common navigation, typography, components, spacing, and visual conventions.

10. Business Rules

- Only authenticated users may access protected placement functionality.
- A user shall operate according to the permissions of the assigned role.
- A job drive should be approved by an administrator before being treated as an approved placement opportunity, according to the agreed workflow.
- Eligibility should consider the job's defined criteria such as minimum CGPA, branch, backlog rules, graduation year, and required skills.
- Only eligible students should be permitted to apply unless an administrator-approved exception rule is introduced.
- Application status changes shall be recorded so that the recruitment history can be monitored.
- Final selection should produce a placement outcome containing the relevant offer/package information when available.
- AI match scores are decision-support information and should not automatically replace recruiter or administrator decisions.
- Analytics should be derived from stored application, interview, and placement records rather than manually entered dashboard totals.

11. System Architecture and Technology Requirements

Layer	Technology / Responsibility
Frontend	ASP.NET Core MVC, Razor Views, HTML, CSS, Bootstrap, JavaScript
Backend	ASP.NET Core Web API, C#
ORM	Entity Framework Core
Database	Microsoft SQL Server
Authentication	ASP.NET Core Identity / role-based authorization
AI / ML	Python, FastAPI, sentence-transformers, scikit-learn and/or spaCy as required
Testing	xUnit, Moq, Swagger, Postman
Version Control	Git and GitHub

11.1 Proposed Solution Structure

AI-Placement-Management/

— AIPlacement.Web/	# ASP.NET Core MVC
— AIPlacement.API/	# ASP.NET Core Web API
— AIPlacement.Core/	# Entities, DTOs, interfaces
— AIPlacement.Infrastructure/	# EF Core, repositories, database
— AIPlacement.AI/	# Python/FastAPI AI service
— AIPlacement.Tests/	# Unit/integration tests
— README.md	

12. User Interface Requirements

The final UI shall use one shared design language across the Student, Company, and Admin modules. The detailed visual design is to be finalized jointly before implementation.

Common UI Element	Requirement
Navigation	Role-aware navigation with common layout structure.
Dashboard	Card-based summary metrics and relevant charts/tables.
Tables	Search, filtering, readable columns, and pagination where needed.
Forms	Consistent labels, validation messages, buttons, and error states.
Status indicators	Clear visual labels for job, application, interview, and placement states.
Responsive behavior	Pages should remain usable across common desktop and smaller screen widths where practical.
Accessibility	Readable contrast, labels, keyboard-friendly controls, and meaningful validation/error messages.

13. Admin + Analytics Requirements

Pair 3 is responsible for the administrative control center and analytics portion of the system. These requirements should be implemented only after the shared database, authentication, API contracts, and UI conventions are agreed upon.

ID	Admin Requirement
A-01	Admin login and role-based authorization.
A-02	View and manage student records.
A-03	View and manage company records.
A-04	View pending job drives and approve or reject them.
A-05	Monitor applications and recruitment outcomes.
A-06	View placement results and selected students.
A-07	Display total students, companies, drives, applications, shortlisted candidates, and selected students.
A-08	Display placement percentage and branch-wise placement analytics.
A-09	Display application statistics by company/job.
A-10	Display average and highest package information.
A-11	Provide reports based on stored placement data.
A-12	Support integration testing across the complete workflow.

14. Testing and Acceptance Criteria

The system shall be considered functionally acceptable when the core end-to-end placement scenario can be demonstrated successfully.

Area	Acceptance Condition
Authentication	Users can log in and are directed to functions appropriate to their role.
Student	A student can maintain a profile, upload a resume, view jobs, and apply to an eligible drive.
AI	A resume can be processed to produce extracted skills and a job compatibility result/skill-gap output.
Company	A recruiter can create a job drive, view applicants, shortlist candidates, and manage interviews.
Admin	An admin can approve/reject drives, monitor applications, view placement outcomes, and manage users.
Analytics	Dashboard statistics and placement analytics are calculated from system data.
Integration	A complete student-to-placement workflow works across MVC, API, database, and AI service boundaries.
Testing	Important business logic and API behavior have corresponding tests and documented results.

15. Assumptions, Constraints, and Risks

15.1 Assumptions

- The college placement process and terminology used by the team are sufficiently represented by the defined Student, Company, Job, Application, Interview, and Placement workflows.
- Users have valid information required to create profiles and applications.
- The AI service can receive the required resume/job data through an agreed API contract.

15.2 Constraints

- The project is developed as a coordinated multi-pair team project using one shared repository and shared application structure.
- Shared files such as database models, common DTOs, _Layout.cshtml, and CSS require coordination before merging.
- The project must remain feasible within the academic project timeline and available development resources.

15.3 Key Risks and Mitigation

Risk	Impact	Mitigation
Database design changes late	High	Finalize ER diagram and review relationships before feature implementation.
Pairs develop incompatible APIs	High	Document API contracts and response formats before coding.
Shared UI conflicts	Medium	Agree on a common design system and coordinate changes to shared files.
AI output is inconsistent	Medium	Treat AI as decision support, validate inputs/outputs, and test representative cases.
Integration fails near deadline	High	Integrate incrementally instead of waiting until all modules are finished.

16. Expected Deliverables

- Approved requirement analysis document.
- ER diagram and database schema.
- ASP.NET Core solution with shared architecture.
- Role-based authentication and authorization.
- Student + AI module.
- Company + Recruitment module.
- Admin + Analytics module.
- REST API documentation / Swagger documentation.
- AI service integration.
- Unit, API, integration, and UI test evidence.
- Project documentation, diagrams, presentation, and final demonstration.

16.1 Final Demonstration Scenario

The final demonstration should show one complete scenario: student registration → profile completion → resume upload → AI analysis → recruiter job creation → eligibility check → AI match score and skill gaps → student application → recruiter review and shortlist → interview → interview result → student selection → updated admin placement statistics.

17. Requirement Traceability Summary

Requirement Group	Primary Module	Supporting Modules
FR-01 to FR-02	Shared Authentication	All
FR-03 to FR-06	Student + AI	Admin
FR-07 to FR-09	Company + Recruitment	Admin
FR-10 to FR-12	Student + AI	Company + Recruitment
FR-13 to FR-16	Company + Recruitment	Student + AI, Admin
FR-17 to FR-22	Admin + Analytics	Student + AI, Company + Recruitment
FR-23	Shared API Layer	All
FR-24	Testing / Integration	All